题解：2026 年大连理工大学大学生程序设计新生赛暨第七届 ICPC 辽宁省大学生程序设计竞赛大连理工大学选拔赛

A. 看你操作！

我们要求将当前数组变为回文数组的最小成本，也就是求到达对称位置相等的目标还差多远。因此，我们考虑求数组 $[a_1 - a_n, a_2 - a_{n-1}, \ldots, a_{\lfloor n/2 \rfloor} - a_{n+1-\lfloor n/2 \rfloor}]$，就可以看到两侧元素需要调整的程度，而我们的目标就是把这个数组中的每个元素全部调整成 0。

而每次操作是对原数组取子数组进行 $+1$ 的操作，分类讨论可以发现，每个操作就相当于对上面的调整后的数组的某个区间进行区间 $+1$ 或 -1 操作，而每个子数组的 $+1$ 或 -1 操作都可以在原数组中找到对应方式。

因此问题彻底转化为对一个数组的子数组进行 $+1$ 或 -1 操作，最少要多少次才能全部变成 0。区间加减用差分数组会更容易理解。我们差分后，相当于对两个位置一个 $+1$，一个 -1，只需配对就能找到对应的区间操作。

因此只需求差分数组中正的元素的和或负的元素的和即可。（这里 $[d_1, d_2, \ldots, d_k]$ 数组的差分理解为 $[d_1, d_2 - d_1, \ldots, d_n - d_{n-1}, 0 - d_n]$，这样才能保证 $+1, -1$ 一一匹配。）

时间复杂度为 $\mathcal{O}(n)$。

B. 【模板】

将数组 A 理解为一个有向图，从 i 向 $A[i]$ 连边。则每个点出度为 1，该图是一个基环树。

而 u 可以到达 v 等价于这个图中 u 可以到达 v。所以我们就相当于从一个点出发不断去可到达的 B 数值更大的点，问最多能走几步。

环上的点，只需查询环上有多少个数比当前的数更大就好。

对于环上延伸出的树结构，情况如何呢？我们可以从环出发，看走到该点的路径。因为是反着走的，所以相当于要求 B 是递减的。我们把环上的数递减排列（相同元素删去）再加上后面路径上的所有数得到一个新数组，我们相当于要求这个数组包含最后一项的最长递减子序列。

这是很典型的问题，我们可以维护长度为 x 的最长递减子序列的最后一项的最大值 $dp[x]$。那么在新考虑一项 v 时，我们二分找到最大的大于 v 的下标 i，则我们可以用 v 更新 $dp[i+1]$。同时包含 v 的最长递减子序列的长度是 $i+1$。

这只解决了单点的情况如何处理。如果存在多条路径呢？我们发现，新增一项时，对 dp 数组的修改是 $\mathcal{O}(1)$ 的，因此删去一个元素，恢复 dp 数组也是 $\mathcal{O}(1)$ 的。

所以，我们可以从环上出发进行 DFS，新遍历到一个点时更新 dp 数组，这个点的搜索结束时，撤销这个点对 dp 数组的修改。这样就保证了我们在遍历到每个结点时，恰好添加了从环到这个结点的路径上的所有元素，也就得到了每个结点的答案。同时对于环上的各个树，初始的数组都是一致的，也可以共用。

时间复杂度为 $\mathcal{O}(n \log n)$。

C. 不太全新的数组

首先，操作可以让整个数组中的每个元素变成同一个 $[1, M]$ 内的整数 x，直接全体关于参数 x 分别进行两类操作即可。

如果数组最后存在不同元素呢？不妨设数组最后的最小值和最大值 v_m, v_M 满足 $v_m < v_M$，则第一类操作使用的数一定不大于数组原本的最小值，第二类操作使用的数一定不小于数组原本的最大值，否则数组所有元素都已经相等了。

因此最后的 v_m, v_M 都得在原本的最小值和最大值之间。而一旦这两个数确定，数组最后的情况也完全确定了。因此只需求满足条件的 (v_m, v_M) 的选取方案数。

时间复杂度为 $\mathcal{O}(n)$。

D. 找到目标合法括号序列

考虑是字典序的第几个，只需要用 1 加上比当前字符串字典序小的字符串个数就好。

为此，我们枚举前缀长度 l，看前 l 项相同，且第 $l+1$ 项比当前字符串严格更小的合法括号字符串个数。这样将所有 l 对应的答案累加就是答案了。

这相当于要求第 $l+1$ 项刚好是右括号，且 $l+1$ 项开始，左右括号数量和原字符串一致，且可以是合法括号序列的后缀，求对应的方案数量。

我们也就将问题转化为，x 个左括号，y 个右括号，可以凑出多少括号序列的后缀。我们将字符串反转，并将左括号变为右括号，右括号变成左括号，从后往前的问题就变为了从前往后的问题，求的就是合法括号序列前缀的数量。也就进一步转化为，x' 个左括号，y' 个右括号，可以凑出多少括号序列的前缀。

为此，我们在平面直角坐标系中构造一个 $x' \times y'$ 的矩形，四个定点分别为 $(0,0), (x',0), (0,y'), (x',y')$，以左下角为起点，将左括号理解为向右走一个单位，将右括号理解为向上走一个单位，则我们要求的就是走到右上角的方案数，同时我们不能越过 $y=x$。

总方案数是 $C_{x'+y'}^{x'}$，不合法的方案数怎么算呢？我们可以考虑 $y=x+1$ 这条直线，一旦触碰到了它就不合法，且不合法的方案一定触碰到了它。我们再将第一次触碰这条直线的那一段折线路径关于 $y=x+1$ 作对称，则得到的新路径起点为 $(-1,1)$。可以发现 $(-1,1)$ 到 (x',y') 的路径与不合法方案一一对应。所以不合法方案数量为 $C_{x'+y'}^{x'+1}$。

综上，这个问题的答案是 $C_{x'+y'}^{x'}-C_{x'+y'}^{x'+1}$，这在预处理阶乘和阶乘的逆元后，可以 $\mathcal{O}(1)$ 得到答案，因此问题解决。

时间复杂度为 $\mathcal{O}(n)$。

E. Wordle

由于三种颜色的处理是有顺序性的，因此我们按照对应顺序进行处理。

首先，我们将两个字符串能匹配上的位置染为蓝色。对于剩下的位置，如何确认猜测的字符是否还在隐藏字符串中出现呢？我们统计隐藏字符串中的剩余字符。接下来顺序遍历猜测字符串中的每个位置，如果该位置能和隐藏字符串匹配，即剩余字符中仍有该字符，则染为黄色，并且将隐藏字符串中统计的剩余字符数量进行更新；否则，染为灰色。

因为只是顺序遍历了字符串常数次，时间复杂度为 $\mathcal{O}(n)$。

F. 骗过检测器

不妨设进行数组分割后，最终每个子数组的异或和都是 x，那么整个数组的异或和只可能是 0 或 x。

因此，若整体异或和非零，则每个子数组的异或和也必须等于该值。此时可以贪心统计结果，从头开始遍历数组，一旦找到可以分割的位置就分割，即可得到最大分段数量。

接下来考虑数组整体异或和为 0 的情况。

区间的异或和相当于求完前缀异或和后，左右两端前缀异或和的异或结果。此时我们将对应位置的前缀异或和取出，也就发现我们要找到形如 $[0,x,0,x,\ldots,0]$ 的最长子序列（这里前缀异或和包含了长度为 0 的前缀异或结果 0）。

一种比较直接的想法是枚举非 0 的 x，从初始位置开始，不断二分找到下一个可以选的前缀异或和为 x 或 0 的位置，直至走到数组末端。因为总二分次数不超过 x 的个数乘 2，所以总复杂度不超过 $\mathcal{O}(n\log n)$。

还有别的计算方法。我们在遍历数组的过程中，记录每个前缀异或和 x 对应的当前答案。只要相邻的两个 x 之间出现了一个 0，对应的 x 的答案就可以增加 2。为此可以记录最后一次出现 0 的位置，以及到上一个 x 为止最后一次出现 0 的位置，只要这两者不同，这两个 x 之间就一定出现了至少一个 0。

于是顺序遍历前缀异或和数组，在遇到非零数时，判断其前一次出现和当前位置之间是否有 0，有的话更新答案即可，同时更新当前数对应的前一次 0 出现位置；遇到 0 时，更新 0 的最近出现位置。这样，只要我们使用哈希表，复杂度就可以优化为 $\mathcal{O}(n)$。

这种情况也可以理解为，用 0 将前缀异或和数组分为多段，如果 x 在其中的 k 段中出现了，那么对应的答案就是 $2k$，这个理解也可以一段段地用哈希表进行统计，也可以达到 $\mathcal{O}(n)$ 复杂度。

如果 x 为 0 呢？直接类似于第一种情况，进行模拟即可，需要用其更新最终答案。

时间复杂度为 $\mathcal{O}(n)$。

G. 后缀的查询

题目所有操作都基于后缀——加入字符串，添加后缀字符，查询最长公共后缀总长度。而如果我们将它们转化为前缀操作，就方便很多了。

于是我们将插入的字符串和查询的字符串反转，添加后缀字符变成添加前缀字符，我们就只需求 LCP（最长公共前缀）的总长了。这件事显然用字典树很容易维护。

字典树中插入字符串是很容易的，而要在所有字符串开头加一个字符，相当于另开一个根，在该根对应的字符位置连接原有的字典树。

时间复杂度为 $\mathcal{O}\left(q+\sum|s_i|\right)$。

H. 大步小步

大步使某个坐标增加 2，不会改变坐标的奇偶性；小步使某个坐标增加 1，且整个行程至多使用一次。

若 x, y 都是奇数，两个方向各需要一次小步，无法到达。若两个坐标都是偶数，只使用大步即可；若恰有一个坐标为奇数，在该方向使用一次小步，再使用大步即可。

因此，当且仅当 x, y 中至少有一个为偶数时，输出 `Yes`；否则输出 `No`。

判断条件（C++）：

```
x % 2 == 0 || y % 2 == 0
```

每组数据的时间复杂度为 $O(1)$，额外空间复杂度为 $O(1)$。